Data Structures

Dilip Maripuri
Associate Professor
Department of Computer Applications

Data Structures

Session Outline

PES UNIVERSITY

Hashing

Data Structures

Hashing

- Hashing is a fundamental technique in computer science that is primarily utilized for rapid data retrieval.

Hashing

- **Definition and Purpose**
  - **Hashing**
    - Involves transforming input data (such as strings, files, or objects) into a compressed, fixed-size output known as a hash code, or simply "hash."
  - **Primary Use**
    - The generated hash is used for quick data indexing and retrieval in various applications.

Hashing

- **Hash Function**
  - **Role**
    - A hash function calculates a hash by processing input data. This hash acts as a unique identifier for the data.
  - **Output**
    - It produces a short, fixed-size value from potentially large or complex input data.

Data Structures

Hashing

- **Hash Table Utilization**
  - **Data Structure**
    - Hashes are primarily used with hash tables, a type of data structure optimized for speedy data access.
  - **Process**
    - **Storage**
      - When data is added to a hash table, the hash function determines the position based on the calculated hash.
    - **Retrieval**
      - To retrieve data, the hash function recalculates the hash to quickly locate the data in the table.

Hashing

- **Efficiency and Performance**
  - **Speed**
    - Hashing allows for near constant time complexity in data retrieval, making it highly efficient irrespective of the data size.
  - **Application**
    - Essential for fast-paced applications needing quick data access, such as database indexing and caching.

Hashing

- **Advantages of Hashing**
  - **Consistency**
    - Each unique piece of data ideally corresponds to a unique hash, reducing retrieval times.
  - **Compact Representation**
    - Hashes provide a compact representation of data, simplifying processes and improving performance in data management systems.

Data Structures

Hashing

- The main purposes of hash functions in computing include
  - **Speed**
    - Hash functions enable very fast data retrieval, almost independent of the size of the dataset.
  - **Uniformity**
    - Good hash functions distribute data evenly across the hash table, minimizing collisions (where different inputs produce the same hash).
  - **Determinism**
    - The same input will always produce the same hash.
  - **Efficiency**
    - Hash functions should be easy to compute while still achieving good distribution and minimal collisions.

Data Structures

Hashing

- A hash function takes an input (or message) and returns a fixed-size string of bytes.

- The output, the hash, typically has a fixed length, and small changes in the input produce unpredictable, large changes in the output.

Hashing

- **Basic Hash Functions**
  - **Division-Remainder Method**
    - A simple approach where the hash is calculated as the remainder of the input divided by the hash table size.
    - For example, for an input `x` and table size `m`, the hash is `h(x) = x mod m`.

Hashing

PES UNIVERSITY

- **Basic Hash Functions - Division-Remainder Method**
  - Suppose you are developing a system to manage the seating arrangement of guests by their ID number at a banquet, and you want to use hashing for quick look-up of where each guest should be seated.
  - The banquet hall has 10 tables, so you decide to use the Division-Remainder Method to determine which table each guest should sit at based on their unique guest ID.

Hashing

- **Basic Hash Functions - Division-Remainder Method**
  - **Hash Function Definition**
    - Let the m be the number of tables => m = 10.
    - Define the hash function as h(x)=x mod m, where x is the guest ID.

Hashing

- **Basic Hash Functions - Division-Remainder Method**
  - **Applying Hash Function**
    - **Guest ID: 23**
      - Calculate hash: $h(23)=23 \bmod 10= 3$
      - Therefore, Guest 23 is assigned to Table 3.
    - **Guest ID: 47**
      - Calculate hash: $h(47)=47 \bmod 10= 7$
      - Therefore, Guest 47 is assigned to Table 7.

Hashing

- **Basic Hash Functions - Division-Remainder Method**
  - By using the Division-Remainder Method, each guest ID is converted into a table number from 0 to 9.
  - This method ensures a simple and quick way to determine the seating without having to check each ID individually against a list.
  - The modulus operation ensures that the output (table number) is always within the range of available tables, regardless of how large the guest ID is.

Hashing

- **Basic Hash Functions**
  - **Multiplication Method**
    - This method involves multiplying the input by a constant, taking the fractional part of the result, and then multiplying by the table size.
    - The hash is then the floor of this value.
    - Its less sensitive to patterns in the data than the division method.

- **Basic Hash Functions - Multiplication Method**
  - Imagine you are designing a system to allocate tasks to different servers based on task IDs, and you want to distribute these tasks evenly across a fixed number of servers using hashing.
  - There are 5 servers available.

Hashing

PES UNIVERSITY

- **Basic Hash Functions - Multiplication Method**
  - **Hash Function Definition**
    - Let the m be the number of servers => m = 5.
    - Choose a constant A which is not too large or too small; typically, it is an irrational number fraction.
    - For simplicity, let's use A =0.6180339887
      - the fractional part of the golden ratio, commonly used in such scenarios.

Hashing

- **Basic Hash Functions - Multiplication Method**
  - **Hash Function Definition**
    - The choice of an irrational number helps in breaking up any regular patterns in the input data because the multiplication of a regular pattern with an irrational number results in outputs that are more spread out over the number line

Hashing

- **Basic Hash Functions - Multiplication Method**
  - **Applying the Hash Function**
    - Task ID: 12345
    - Calculate hash
      - Multiply the input by the constant $12345 \times 0.6180339887 \approx 7629.629590501$
    - Take the fractional part of the result
      - $7626.629590501 - 7626 = 0.629590501$
    - Multiply by the table size
      - $0.629590501 \times 5 \approx 3.147952505$
    - Take the floor of this value to get the hash

      $\text{floor}(3.147952505) = 3$
    - Therefore, Task 12345 is assigned to Server 3.

Hashing

- **Basic Hash Functions - Multiplication Method**
  - This focus on the fractional part is key to why the Multiplication Method is effective at distributing entries evenly.
  - By eliminating the integral part of the product, the method prevents the original magnitude of inputs from influencing their hash values directly.

Hashing

- **Basic Hash Functions - Multiplication Method**
  - This is particularly beneficial when the input data may have existing numerical or structural patterns that could otherwise lead to clustering in certain slots of a hash table.
  - For instance, if task IDs are sequential or have specific numeric patterns, the Multiplication Method helps in ensuring these patterns do not translate directly to clustering in the hash table.

Hashing

- **Basic Hash Functions - Multiplication Method**
  - The Multiplication Method leverages mathematical properties to optimize the hashing process, ensuring that data is distributed evenly and that existing patterns in the data do not skew the distribution.

Hashing

- **Basic Hash Functions**
  - **Folding Method**
    - This method splits the input into several parts, adds them together, and then takes the modulo of the result with the table size.
    - Its useful for hashing functions where the input might be significantly longer than the desired hash outputs.

Hashing

- **Basic Hash Functions - Folding Method**
  - Imagine you need to hash phone numbers to assign customers to different service queues based on their number.
  - You have 10 service queues, so you decide to use the Folding Method to determine the queue assignment.

Data Structures

Hashing

- **Basic Hash Functions - Folding Method**
  - **Hash Function Setup**
    - **Number of Queues (m): 10**
      - so we will use mod 10 in our hash function.
    - **Input - Phone Number**
      - Suppose we use the phone number 123-456-7890.

Hashing

- **Basic Hash Functions - Folding Method**
  - **Steps for Hashing**
    - **Splitting the Input**
      - Break the phone number into segments of equal length: 123, 456, 789, 0 (the last segment can be shorter).
    - **Converting Segments**
      - Convert these segments into integers: 123, 456, 789, 0.
    - **Adding the Segments**
      - Sum these numbers: 123 + 456 + 789 + 0 = 1368.
    - **Applying Modulo**
      - Take the modulo of the sum with the table size: 1368 mod 10 = 8

Compiled by Dilip Maripuri

Hashing

- **Basic Hash Functions - Folding Method**
  - **Queue Assignment**
    - Based on the hash, the customer with the phone number 123-456-7890 would be assigned to Queue 8.

Hashing

- **Basic Hash Functions**
  - **Mid-square Method**
    - This involves squaring the input, then extracting some portion of the resulting number (often the middle part) as the hash.
    - It can be effective but depends heavily on the input data characteristics.

Hashing

PES UNIVERSITY

- **Basic Hash Functions - Mid-square Method**
  - Imagine we are tasked with assigning parking spaces to cars in a large parking garage.
  - Each car has a unique identification number, and we want to use hashing to determine which parking spot to assign each car efficiently.
  - We decide to use the Mid-square Method to generate a hash code from each car's ID to determine the parking spot number.

Hashing

- **Basic Hash Functions - Mid-square Method**
  - **Step-by-Step Process**
    - **Hash Function Definition**
      - We define the hash function to take a car ID, square it, and then extract a part of the resulting number as the hash.

Data Structures

Hashing

- **Basic Hash Functions - Mid-square Method**
  - **Step-by-Step Process**
    - **Applying the Hash Function**
      - Suppose the parking garage has 100 spots, so we need a two-digit result from our hash function.
      - Car ID: 1234
    - **Calculation**
      - Square the ID: $1234^2 = 1522756$
    - **Extract the middle two digits** to form the hash: from 1522756, take 27.
      - So, the hash (parking spot number) for Car ID 1234 is 27.

Hashing

- **Basic Hash Functions - Mid-square Method**
  - Squaring the input (Car ID) generates a larger number, which helps in deriving a more distributed range of hash values.
  - The extracted middle digits (hash) decide the parking spot.
  - This method utilizes the middle part of the square of the input, which typically contains enough randomness from the original input to minimize collisions in a small hash table like our 100 parking spots.

Hashing

- **Basic Hash Functions - Mid-square Method**
  - This hashing technique, while dependent on the nature of input values, provides an easy and effective way to distribute items (in this case, cars) across a fixed number of categories (parking spots), ensuring an efficient use of space and quick retrieval based on ID numbers.

Hashing

- **Basic Hash Functions - Mid-square Method**
  - This hashing technique, while dependent on the nature of input values, provides an easy and effective way to distribute items (in this case, cars) across a fixed number of categories (parking spots), ensuring an efficient use of space and quick retrieval based on ID numbers.

Hashing

- **Characteristics of Effective Hash Functions**
  - **Efficient Computability:** It should be quick to compute the hash value for any given input.
  - **Uniform Distribution:** The hash function should distribute data uniformly across the entire hash table to minimize clustering.
  - **Minimized Collisions:** While collisions (where two inputs hash to the same output) are inevitable in hashing, a good hash function minimizes their frequency.

Hashing

- **Characteristics of Effective Hash Functions**
  - **Avalanche Effect:** A small change in the input should cause a significant change in the output hash, making the output appear random and uniformly distributed.
  - **Irreversibility:** Especially in cryptographic hash functions, it should be computationally infeasible to reverse the hash function to find the original input.

Hash Functions - Summary

Basic Concept	
Method	**Description**
Division-Remainder Method	Uses the modulus operator to divide the input by the table size and take the remainder as the hash.
Multiplication Method	Multiplies the input by a constant, takes the fractional part of the result, multiplies this by the table size, and uses the integer part as the hash.
Folding Method	Splits the input into several parts, sums them, and then uses the modulus of this sum as the hash.
Mid-square Method	Squares the input, extracts a portion of the resulting number (often the middle part) to use as the hash.

Data Structures

Hash Functions - Summary

Dependence on Input Size	
Method	**Description**
Division-Remainder Method	Directly dependent on the input size; the choice of table size (modulus value) is critical to distribution quality.
Multiplication Method	Less sensitive to the size or pattern of the input due to the involvement of a constant multiplier.
Folding Method	Independent of input size as long as the segments and fold size are appropriately chosen.
Mid-square Method	Highly dependent on the input data characteristics; effectiveness varies greatly based on how input values distribute when squared.

Hash Functions - Summary

Uniform Distribution	
Method	**Description**
Division-Remainder Method	Can be poor if the table size is not chosen well (e.g., not a prime number).
Multiplication Method	Typically provides good distribution, especially if the constant is chosen to promote incoherence between the data and the table size.
Folding Method	Reasonably good if the input is random and segments are appropriately combined.
Mid-square Method	Distribution can be uneven, depending on how digits are extracted from the squared result.

Hash Functions - Summary

Computational Efficiency	
Method	**Description**
Division-Remainder Method	Very efficient; simple modulus operation is all that is required.
Multiplication Method	Moderately efficient; involves multiplication and floating-point operations, which are costlier than simple modulus.
Folding Method	Can be less efficient due to multiple additions and possibly more complex data handling.
Mid-square Method	Requires more computation (squaring and digit extraction), making it potentially the least efficient of the four.

Data Structures

Hash Functions - Summary

Collision Handling	
Method	**Description**
Division-Remainder Method	Collisions likely if the modulus is not well chosen or data is not uniformly distributed.
Multiplication Method	Collisions are generally fewer due to better spread, especially with a well-chosen multiplier.
Folding Method	Collisions depend on how the data segments overlap and are combined.
Mid-square Method	Potential for more unique collisions due to reliance on certain parts of the squared number.

Data Structures

Hash Functions - Summary

Ease of Implementation	
Method	**Description**
Division-Remainder Method	Very easy to implement.
Multiplication Method	Moderate difficulty due to the need to manage fractional parts and constants.
Folding Method	Moderately complex, requires careful handling of data segments.
Mid-square Method	More complex due to the need for squaring and digit extraction procedures.

Data Structures

Hash Functions - Summary

Typical Use Cases	
Method	**Description**
Division-Remainder Method	Good for quick, simple hashing when the data set is not too large and is well-distributed.
Multiplication Method	Suitable for scenarios where the data set might not be uniformly distributed; often used in real-time systems where good distribution is critical.
Folding Method	Useful when the input data is large and needs to be condensed in a controlled manner.
Mid-square Method	Effective in scenarios where the characteristics of input data are known and can be exploited for better hash distribution.

Data Structures

Hash Functions - Summary

When to Use	
Method	**Description**
Division-Remainder Method	Best used when the dataset is static or grows predictably, and the modulus (table size) can be chosen to minimize collisions effectively.
Multiplication Method	Best when the dataset requires a hash function that can deal with non-uniform data distributions or in high-performance environments where every bit of efficiency counts.
Folding Method	Ideal for handling large inputs or when the data can naturally be divided into segments, such as strings or large numbers.
Mid-square Method	Preferable in applications where the hash quality directly influences the outcome, such as cryptographic applications or data analysis.

Data Structures

Hashing

- Hash functions are crucial for efficient data management and security in computer science.

- They are designed to maximize efficiency in storing and retrieving data, ensuring security in data transmission, and providing integrity in various information processing tasks.

- The choice of a hash function depends largely on the specific requirements of the application, including the nature of the data, the environment in which it operates, and the balance between speed and security.

THANK YOU

Dilip Maripuri
Associate Professor
Department of Computer Applications
dilip.maripuri@pes.edu